Module 11: Caching and Transaction Management in Spring Boot

Questions

Q1. What is caching, and why is it important in a Spring Boot application?

Answer: Caching is a technique that stores copies of frequently accessed data in a faster storage layer so future requests can be served without repeating the expensive operation that produced the data. In a Spring Boot application, caching reduces load on databases, external APIs, and CPU-intensive computations, which improves response times and lowers infrastructure cost. For example, instead of querying a slow report table every time a dashboard loads, the result can be cached for a few minutes. The trade-off is complexity: cached data can become stale, and cache invalidation must be handled carefully. Caching is most effective for read-heavy, stable data and least effective for data that changes constantly.

Q2. What is the Spring Cache abstraction, and how do you enable it?

Answer: The Spring Cache abstraction provides a consistent set of annotations such as `@Cacheable`, `@CachePut`, `@CacheEvict`, and `@Caching` that work with different cache providers. You enable it by adding `@EnableCaching` to a configuration class and including a cache provider dependency such as Caffeine or Redis. Spring Boot auto-configures a default cache manager when it detects a provider on the classpath. The abstraction lets you switch from an in-memory cache to a distributed cache without changing business code, because the annotations are provider-agnostic. A common mistake is enabling caching but not configuring a provider, which causes Spring to use a simple concurrent map that may not behave the same in production.

Q3. What does `@Cacheable` do, and how do you use it?

Answer: `@Cacheable` marks a method whose result should be stored in the cache. Before the method runs, Spring checks the cache for an entry matching the key; if found, it returns the cached value without executing the method. If not found, it executes the method and stores the result. You specify the cache name with `value` or `cacheNames`, and you can customize the key with `key` using SpEL expressions. For example, `@Cacheable(value = "products", key = "#id")` caches

products by ID. A common pitfall is caching a method whose arguments or return value are not serializable when using Redis, which causes runtime errors.

Q4. What is the difference between `@Cacheable` and `@CachePut`?

Answer: `@Cacheable` reads from the cache first and only executes the method if the cache miss occurs. `@CachePut` always executes the method and then updates the cache with the returned value. `@CachePut` is useful after a create or update operation when you want to refresh the cached entry without waiting for the next read. For example, after updating a product, you can use `@CachePut(value = "products", key = "#product.id")` to refresh the cache immediately. The key difference is that `@Cacheable` can skip method execution, while `@CachePut` never does.

Q5. What is `@CacheEvict`, and when would you use it?

Answer: `@CacheEvict` removes entries from the cache. It is used when data changes and the cached copy is no longer valid. You can evict a single entry by key, for example when a product is updated, or all entries in a cache using `allEntries = true`. For example, `@CacheEvict(value = "products", key = "#id")` removes the cached product with the given ID. Eviction is critical because stale cache data is one of the most common production bugs. A common mistake is updating the database without evicting or refreshing the corresponding cache entry, which makes subsequent reads return old data.

Q6. What is a cache key, and how do you build a good one?

Answer: A cache key is the identifier used to store and retrieve a cached entry. Spring generates a default key from the method parameters, but for complex cases you can build a custom key using SpEL. A good key is unique, stable, and deterministic for the same input, and it should not include values that change on every request such as timestamps or random IDs. For example, `@Cacheable(value = "orders", key = "#customerId + '_' + #status")` creates a readable and consistent key. Bad keys lead to cache collisions or duplicate entries. When using Redis, keep keys reasonably short and avoid unbounded values such as user names.

Q7. What are the default caching options in Spring Boot, and when is each appropriate?

Answer: The simplest default is the `ConcurrentMapCacheManager`, which stores entries in memory using a `ConcurrentHashMap`. It is fine for local development and single-instance applications but does not share state across instances and has no eviction policies. Caffeine is a popular in-process cache with size and time-based eviction, and Spring Boot auto-configures it when `caffeine` is on the classpath. Redis is a distributed cache that shares state across multiple application instances and supports TTL and eviction. Choose the in-memory option for simple local caching and Redis when running multiple instances or needing persistence and shared invalidation.

Q08. How do you configure Redis caching in Spring Boot?

Answer: You add the `spring-boot-starter-data-redis` dependency and set `spring.cache.type=redis` along with `spring.redis.host` and `spring.redis.port`. Spring Boot auto-configures a `RedisCacheManager` that serializes cache entries using the configured Redis serializer, usually `JdkSerializationRedisSerializer` by default. You can customize TTL, cache names, and serialization through `CacheProperties` or by defining a `RedisCacheConfiguration` bean. For example, you can set a default expiration of 10 minutes and per-cache overrides. A common issue is using default Java serialization, which is brittle and slow; many teams switch to JSON serialization with Jackson or `GenericJackson2JsonRedisSerializer`.

Q09. What are cache TTL and eviction policies, and why do they matter?

Answer: Time-to-live, or TTL, is the duration after which a cache entry expires automatically. Eviction policies decide which entries to remove when the cache reaches its maximum size, such as least recently used, least frequently used, or first-in-first-out. TTL prevents stale data from staying in the cache forever, while eviction policies prevent unbounded memory growth. They matter because a cache without TTL can serve outdated information for hours or days, and a cache without eviction can consume all available memory. In Redis you configure TTL per key or per cache; in Caffeine you configure both TTL and maximum size.

Q10. What is the cache-aside pattern, and how does Spring implement it?

Answer: Cache-aside means the application is responsible for checking the cache, loading data on a miss, and updating or evicting entries. Spring's `@Cacheable` follows this pattern by checking the cache before method execution and storing the result on a miss. The application remains in control, which is simple but requires developers to remember to invalidate the cache when data changes. An alternative is write-through or read-through caching, where the cache itself manages data loading, but

Spring annotations are built around the cache-aside model. The main risk of cache-aside is inconsistency if an update forgets to evict the cache.

Q11. What is a database transaction, and what are the ACID properties?

Answer: A database transaction is a logical unit of work that groups one or more database operations so that they either all complete successfully or none of them take effect. In a Spring Boot application backed by a relational database, transactions are essential whenever a business operation touches multiple rows or tables, because partial updates can leave the database in an inconsistent state. ACID is the set of guarantees that make transactions reliable in multi-user systems. Atomicity means the transaction is treated as a single indivisible unit: either every operation commits, or the entire transaction is rolled back to the state before it began. If a bank transfer debits one account but fails to credit another, atomicity ensures the debit is undone so money does not disappear. Consistency means every committed transaction leaves the database in a valid state that satisfies all defined rules, constraints, triggers, and cascades. For example, if a foreign key constraint requires an order to have a valid customer, a transaction that inserts an order without a customer must fail. Isolation means concurrently executing transactions behave as if they were running one after another, even though they may actually run at the same time. Isolation prevents one transaction from seeing uncommitted changes from another and stops concurrent updates from corrupting each other's results. Durability means once a transaction commits, its changes are permanent even if the database server crashes immediately afterward, because the changes have been written to a persistent medium such as a transaction log or disk. In production, losing any of these properties causes real problems: without atomicity, partial payments or half-completed orders occur; without consistency, invalid data enters reports and downstream systems; without isolation, users see each other's unfinished work; and without durability, committed data can vanish after a crash.

Q12. What are the common transaction isolation levels, and what problems do they prevent?

Answer: The four standard isolation levels are READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, and SERIALIZABLE. READ UNCOMMITTED allows dirty reads, where a transaction sees uncommitted changes from another transaction. READ COMMITTED prevents dirty reads but allows non-repeatable reads, where the same query returns different results within a transaction. REPEATABLE READ prevents non-repeatable reads but may allow phantom reads, where new rows appear between queries. SERIALIZABLE prevents all three problems by executing transactions as if they were sequential, but it is the slowest. The choice is a trade-off between consistency and concurrency.

Q13. What are dirty read, non-repeatable read, and phantom read?

Answer: A dirty read happens when a transaction reads data that another transaction has modified but not yet committed; if the other transaction rolls back, the read data was never valid. A non-repeatable read happens when a transaction reads the same row twice and gets different values because another transaction committed a change between the reads. A phantom read happens when a transaction runs the same query twice and gets different sets of rows because another transaction inserted or deleted rows that match the query. Understanding these phenomena is essential for choosing the right isolation level and for debugging data inconsistency issues in production.

Q14. How does Spring Boot manage transactions with `@Transactional`?

Answer: Spring Boot uses declarative transaction management powered by AOP. When you annotate a method or class with `@Transactional`, Spring creates a proxy that starts a transaction before the method, commits it on success, and rolls it back on runtime exceptions. By default, rollback happens only for unchecked exceptions and `Error`, not checked exceptions. The transaction boundary is usually the service layer, where business logic orchestrates multiple repository calls. A common mistake is annotating a private method, which cannot be proxied, or calling a `@Transactional` method from within the same class, which bypasses the proxy through self-invocation.

Q15. What is the default propagation behavior of `@Transactional`?

Answer: The default propagation is `REQUIRED`. If a transaction already exists, the method joins it. If no transaction exists, Spring creates a new one. This is the right choice for most service methods because it ensures that multiple operations within a business transaction are atomic. Other propagation levels include `REQUIRES_NEW`, which suspends the existing transaction and creates a new one; `SUPPORTS`, which participates if a transaction exists but runs non-transactionally otherwise; `NOT_SUPPORTED`, which suspends any existing transaction; `MANDATORY`, which requires an existing transaction; `NEVER`, which forbids a transaction; and `NESTED`, which creates a savepoint within an existing transaction. Misusing propagation can cause unexpected rollbacks or data inconsistency.

Q16. What is the difference between `REQUIRED` and `REQUIRES_NEW` transaction propagation?

Answer: `REQUIRED` joins the existing transaction if one exists, so the method participates in the caller's transaction and shares the same commit or rollback outcome. `REQUIRES_NEW` always creates a fresh transaction, suspending any existing transaction for the duration of the method. This is useful when you want a sub-operation to commit independently of the main transaction, such as logging an audit event even if the main transaction rolls back. The trade-off is that `REQUIRES_NEW` is more expensive and can lead to complex transaction states if overused. A common pitfall is assuming a `REQUIRES_NEW` method will roll back when the caller rolls back; it will not, because they are separate transactions.

Q17. How does `@Transactional` handle rollback, and what is the default behavior for checked exceptions?

Answer: By default, `@Transactional` rolls back the transaction only on unchecked exceptions, which are subclasses of `RuntimeException`, and on `Error`. Checked exceptions do not trigger a rollback unless you set `rollbackFor` explicitly, for example `@Transactional(rollbackFor = Exception.class)`. This default surprises many developers when a method throws a checked exception and the database changes are committed. You can also use `noRollbackFor` to prevent rollback for specific runtime exceptions. I always review exception handling in transactional code and make the rollback policy explicit if the service throws checked exceptions.

Q18. What is optimistic locking, and how is it implemented with JPA?

Answer: Optimistic locking assumes that conflicts are rare and does not lock the database row while reading. Instead, it uses a version column, annotated with `@Version`, that is incremented on every update. When a transaction tries to update a row, it checks whether the version still matches the value read earlier. If another transaction changed the row in between, the version differs and JPA throws an `OptimisticLockException`. The application then retries or reports the conflict. Optimistic locking works well for read-heavy environments with infrequent writes because it avoids the overhead of database locks. The version column must be included in the entity and never modified manually.

Q19. What is pessimistic locking, and when should you use it?

Answer: Pessimistic locking acquires a database lock on a row or table for the duration of the transaction, preventing other transactions from modifying the data. In JPA, you can request pessimistic locking with `@Lock(LockModeType.PESSIMISTIC_WRITE)` on a query, or with `findById(id, LockModeType.PESSIMISTIC_WRITE)`. It is useful when contention is high and the cost of an optimistic lock failure and retry is greater than the cost of holding the lock, such as during inventory

decrement or seat reservation. The downside is reduced concurrency and a higher risk of deadlocks. Pessimistic locking should not be used for read-mostly data because it unnecessarily blocks other transactions.

Q20. What is the difference between `LockModeType.PESSIMISTIC_READ` and `PESSIMISTIC_WRITE`?

Answer: `PESSIMISTIC_READ` acquires a shared lock that prevents other transactions from writing the row but allows them to read it. `PESSIMISTIC_WRITE` acquires an exclusive lock that prevents other transactions from both reading and writing the row until the lock is released. The exact behavior depends on the database; some databases map both to the same exclusive lock. For most update scenarios, `PESSIMISTIC_WRITE` is the right choice because it guarantees that the current transaction can modify the row without conflicts. `PESSIMISTIC_READ` is rarely needed and mostly used when you want to prevent lost updates while allowing concurrent reads.

Q21. How do you handle `OptimisticLockException` in a Spring Boot application?

Answer: When an `OptimisticLockException` occurs, the transaction is rolled back and the entity manager becomes invalid, so you cannot retry in the same transaction. The correct approach is to catch the exception at the controller or service facade level, fetch the latest version of the entity, reapply the business changes on the fresh data, and attempt the update again. A retry library such as Spring Retry can automate this with exponential backoff. You should also decide how many retries are reasonable and what to do if all retries fail, such as returning a conflict response to the user. The key is that retrying must happen outside the original transaction.

Q22. What is the difference between programmatic and declarative transaction management?

Answer: Declarative transaction management uses `@Transactional` annotations and AOP proxies, so the transaction boundary is declared and the framework handles it. Programmatic transaction management uses `TransactionTemplate` or the `PlatformTransactionManager` directly, giving the developer full control over when to begin, commit, or roll back. Declarative management is simpler and sufficient for most cases. Programmatic management is needed when the transaction boundary does not align with a single method, such as when committing inside a loop or handling partial failures. A common hybrid is declarative for normal cases and programmatic for exceptional complex flows.

Q23. What is a transaction timeout, and how do you set it?

Answer: A transaction timeout limits how long a transaction can run before it is rolled back. You can set it with `@Transactional(timeout = 30)`, which means the transaction must complete within 30 seconds. This prevents long-running transactions from holding locks and consuming resources indefinitely. Timeouts are especially important for batch jobs or reports that may run unexpectedly long. The timeout applies to the entire transaction, not just a single query, so a method that calls many slow operations can exceed the limit even if each query is fast. I set timeouts based on the expected duration of the business operation and monitor for timeout failures in production.

Q24. How do caching and transactions interact in a Spring Boot application?

Answer: Caching and transactions can interact in tricky ways. If a method caches a value and then the transaction rolls back, the cached value may represent data that was never committed. If a method updates the database and evicts the cache, but the transaction rolls back, the cache remains evicted and the next read reloads from the database, which is safe but slower. To keep them consistent, cache writes should happen after the transaction commits, using `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)` or by separating cache updates from transactional methods. Another option is to use a transactional outbox pattern where cache invalidation messages are written to the database in the same transaction and processed by a background worker.

Scenario based questions

S1. A dashboard loads slowly because it queries a large report table every time. How do you use caching to improve it?

Answer: I first confirm that the report data changes infrequently enough to cache, for example hourly or daily. I add a `@Cacheable` annotation to the service method that builds the dashboard data, using a stable cache key such as the report date or a tenant identifier. I choose a cache provider based on the deployment: Caffeine for a single instance, Redis for multiple instances. I set a TTL that matches the business update frequency so users see fresh data without overwhelming the database. I also add a cache warm-up or background refresh so the first user after TTL expiry does not suffer a slow load. After deploying, I compare the database query count and dashboard response time before and after caching to verify the improvement.

S2. After updating a product in the admin panel, users still see the old price for several minutes. How do you fix the stale cache?

Answer: The update method is probably missing a `@CacheEvict` or `@CachePut` annotation, or the TTL is too long. I add `@CacheEvict(value = "products", key = "#product.id")` to the update method so the cached entry is removed when the product changes. The next read will fetch from the database and repopulate the cache. If the product has related cached data, such as a category page or a search index, I may need to evict those caches too, possibly with `allEntries = true` if the relationship is complex. I also verify that the cache key used for eviction matches the key used by `@Cacheable` exactly, including argument names and serialization. Mismatched keys are a common reason stale data persists.

S3. A `@Cacheable` method works fine locally with Caffeine but throws serialization errors when moved to Redis. Why?

Answer: Caffeine stores objects in memory as Java references, so it does not serialize the return values. Redis stores data as bytes and requires the cached value to be serializable. If the return object does not implement `Serializable` and the default `JdkSerializationRedisSerializer` is being used, the cache put operation fails. The fix is to either make the object serializable, switch to a JSON-based serializer such as `GenericJackson2JsonRedisSerializer`, or use a DTO that is designed for serialization. I usually prefer JSON for shared caches because it is readable, language-agnostic, and avoids the fragility of Java serialization. I also ensure that any nested objects are serializable and configure the serializer in `RedisCacheConfiguration`.

S4. Two microservices share a Redis cache, and one service reads data written by another in an incompatible format. How do you resolve it?

Answer: The root cause is inconsistent serialization or cache key conventions across services. I first agree on a common data format, usually JSON with a well-defined schema or shared DTO classes, and a consistent cache key naming convention such as `service:entity:id`. Each service should use the same Redis serializer and key builder. If the services cannot share classes, I define a shared schema document and validate cache payloads against it. I also namespace cache names by service or bounded context to prevent accidental collisions. In extreme cases, I create a dedicated cache per service or use an event-driven approach where each service maintains its own cache from domain events.

S5. A method annotated with `@Transactional` commits partial data when an exception is thrown. Why?

Answer: The exception is probably a checked exception, and by default `@Transactional` does not roll back for checked exceptions. Another possibility is that the exception is caught inside the method before it reaches the Spring proxy, so the proxy never sees it. I check the exception type and either change it to a runtime exception or add `rollbackFor = Exception.class` to the annotation. If the exception is swallowed in a try-catch block, I rethrow it or mark the transaction for rollback manually with `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`. I also verify that the method is being called through the Spring proxy and not through self-invocation, because self-invocation bypasses the transaction advice entirely.

S6. A service method calls another `@Transactional` method in the same class, but the second method runs without a transaction. Why?

Answer: This is the self-invocation problem. Spring applies `@Transactional` through a proxy, so internal method calls within the same bean bypass the proxy and run without transactional advice. To fix it, I move the transactional method into a separate bean and inject it, or I inject the same bean into itself so calls go through the proxy. Another option is to use `AopContext.currentProxy()` to get the current proxy and call the method through it. The cleanest long-term solution is to refactor the code so that transactional responsibilities are in their own service. I also add an integration test that verifies transactional behavior, because unit tests that instantiate the class directly will not catch this issue.

S7. A batch job processes thousands of records in a single `@Transactional` method and runs out of memory. How do you fix it?

Answer: A single long transaction that accumulates thousands of entities in the persistence context causes memory growth because Hibernate holds every loaded and modified entity until commit. I refactor the job to process records in chunks and commit each chunk in a separate transaction. I can use `REQUIRES_NEW` propagation inside a loop or use `TransactionTemplate` to control commits programmatically. After each chunk, I clear the Hibernate session with `entityManager.clear()` to detach processed entities. I also add pagination so the job reads only a chunk at a time rather than loading all records. The trade-off is that the job is no longer one atomic transaction, so I add idempotency or a state machine so interrupted jobs can resume safely.

S8. A financial transfer between two accounts loses consistency when two transfers happen at the same time. How do you handle it?

Answer: This is a classic concurrency problem where simultaneous transfers can cause incorrect balances due to lost updates. I wrap the transfer logic in a `@Transactional` method and use pessimistic locking to lock both account rows before updating them, for example with `findById(id, LockModeType.PESSIMISTIC_WRITE)`. Locking both rows in a consistent order prevents deadlocks. If contention is low, optimistic locking with a `@Version` column can also work, but it requires retry logic when an `OptimisticLockException` occurs. I also validate balances within the same transaction and ensure the operation is atomic. The key is that the read, check, and update must happen inside a single protected transaction.

S9. A method with `@Transactional(readOnly = true)` is still writing to the database. Why?

Answer: `readOnly` is a hint to the transaction manager and the underlying data access layer that the transaction should not modify data. However, it is not enforced by the database or Spring at the method level. If the code inside calls a repository save or update, the write still happens. Some JDBC drivers and Hibernate can optimize read-only transactions by flushing less aggressively, but they do not block writes. To enforce read-only access, I review the method to remove write operations, or I move read and write logic into separate methods with appropriate transaction settings. I also set `readOnly = true` on query methods as a best practice so the team knows the intended behavior.

S10. An `OptimisticLockException` keeps happening during order updates even with low traffic. What could be wrong?

Answer: Persistent optimistic lock failures usually mean the entity is being loaded and held across multiple requests or that background processes are updating the same rows. I check whether the entity is stored in the HTTP session or a cache and reused across requests, which causes the version to become stale. I also check for scheduled jobs or event listeners that update order status without checking the version. If the entity is loaded in one transaction and updated in another, the version check will fail. The fix is to reload the entity at the start of the update transaction, apply changes to the fresh instance, and rely on JPA's automatic version increment. If conflicts are genuinely frequent, pessimistic locking or a queue-based update model may be more appropriate.

S11. A service uses `@Transactional` on a method that calls an external REST API, and the transaction times out during API slowness. How do you fix it?

Answer: Holding a database transaction while waiting for an external API is a bad practice because it keeps database locks and connections open for an unpredictable duration. I move the external API call outside the transactional method, for example by calling the API first and then starting the transaction to save the result, or by saving a pending record, committing, calling the API, and updating the record in a separate transaction. If the API call must happen inside the business flow, I use `@Transactional(timeout = ...)` or reduce the API client timeout to fail fast. The broader fix is to redesign the flow so that long-running external calls do not happen inside database transactions.

S12. A cache eviction runs after a database update, but the cache is not cleared because the transaction rolled back. How do you make cache eviction reliable?

Answer: If cache eviction happens inside a method that later rolls back, the database changes are undone but the cache is already evicted, which is safe but slower. The more dangerous case is writing to the cache inside a transactional method and then rolling back, leaving stale data in the cache. To make eviction reliable, I move cache writes and evictions to after the transaction commits, using `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)` . This ensures cache changes happen only for committed data. Another approach is the transactional outbox pattern: write an invalidation event to an outbox table in the same transaction, and process it asynchronously to evict the cache after commit.

S13. A method with `REQUIRES_NEW` propagation is supposed to log an audit regardless of the main transaction outcome, but the audit is missing when the main transaction fails. Why?

Answer: If the audit method is called through self-invocation, the `REQUIRES_NEW` annotation is ignored and the audit runs in the main transaction, which rolls back along with the main work. The fix is to move the audit method into a separate bean so the call goes through the Spring proxy and a new transaction is created. I also verify that the audit method does not throw an exception that propagates back to the caller, because an unchecked exception in the audit method can cause the new transaction to roll back. To make the audit truly independent, I catch and log audit exceptions within the audit method so they never affect the caller.

S14. A report query returns inconsistent results when run concurrently with updates. Which isolation level and strategy should you use?

Answer: If the report sees partially committed data or changing values between rows, the isolation level is probably READ COMMITTED, which allows non-repeatable reads and phantom reads. For a consistent snapshot, I use REPEATABLE READ or a read-only transaction if the database supports consistent snapshots, or I run the report against a read replica where write locks do not affect it. If the report must run on the primary database and consistency is critical, SERIALIZABLE is an option but can hurt concurrency. Another approach is to run the report on data as of a specific timestamp using database snapshot features such as PostgreSQL's `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE` or Oracle's flashback query.

S15. A pessimistic lock causes frequent deadlocks in a high-contention workflow. How do you resolve it?

Answer: Deadlocks happen when two transactions acquire locks in different orders. I first ensure that all code paths lock resources in a consistent order, such as always locking the account with the lower ID first. If consistent ordering is not possible, I reduce the lock scope by shortening the transaction duration and moving non-essential work outside the transaction. I also consider switching to optimistic locking with retry logic if conflicts are not extremely frequent. If contention is unavoidable, I introduce a queue or saga pattern so only one process updates the resource at a time. Monitoring deadlock exceptions in logs and database metrics helps identify the offending code paths.

S16. A cached list of products grows without bound and exhausts memory. How do you control it?

Answer: An unbounded cache is dangerous because every unique query key adds a new entry. I configure a maximum size for the cache, for example `spring.cache.caffeine.spec=maximumSize=1000,expireAfterWrite=10m`. In Redis, I set a TTL so old entries expire automatically. I also review whether the cache key contains high-cardinality values such as timestamps or user IDs, which create a new entry for every request. If the list is large, I consider paginating the result and caching individual pages instead of the entire list. For very large datasets, caching becomes less effective and a search index or materialized view may be a better solution.

S17. A `@Cacheable` method is invoked inside another `@Cacheable` method in the same bean, and the inner cache is never used. Why?

Answer: This is the self-invocation problem applied to caching. When the outer method calls the inner method directly on the same bean, the call bypasses the Spring proxy, so the `@Cacheable` advice on the inner method is not applied. The inner method runs every time without checking the cache. To fix it, I move the inner cached method into a separate bean and inject it, or I inject the same bean into itself so calls go through the proxy. I also consider whether both methods really need caching; sometimes only the outer call should be cached, and the inner call should always fetch fresh data.

S18. A method writes to the cache and then throws an exception, leaving the cache inconsistent with the database. How do you prevent this?

Answer: Writing to the cache before the database is committed is risky because a later failure leaves stale cache data that may never expire. The fix is to write to the cache only after the transaction commits, using `@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)`. Alternatively, I separate cache updates from transactional business methods and handle them in a non-transactional layer that runs after success is confirmed. If eventual consistency is acceptable, I use a short TTL so any inconsistency resolves quickly. The safest pattern is cache-aside with database as the source of truth and cache invalidation triggered by committed changes.

S19. A Spring Boot application starts with a `NoCacheManagerBeanFoundException`. What do you check?

Answer: This exception means Spring cannot find a `CacheManager` bean. I first verify that `@EnableCaching` is present on a configuration class. Then I check the classpath for a cache provider dependency such as Caffeine, Redis, or EhCache. If multiple providers are present, Spring may get confused unless I specify `spring.cache.type`. If no provider is present, Spring Boot does not auto-configure a cache manager. I also check for bean naming conflicts or a custom `CacheManager` that failed to initialize. In a test context, I may need to add a test cache manager or disable caching with `spring.cache.type=NONE`.

S20. A transaction rolls back unexpectedly after a method catches and handles an exception. Why?

Answer: Even if the method catches the exception and returns normally, the transaction may already be marked rollback-only by a deeper layer. For example, if a repository call throws a `DataIntegrityViolationException` that is caught, Spring may have already set the transaction status to rollback-only. When the outer method tries to commit, the commit fails. To handle this, I catch the exception and decide whether to continue or explicitly roll back using `TransactionAspectSupport.currentTransactionStatus().setRollbackOnly()`. If the method intends to recover from the exception and commit, I need to ensure the failure happens in a separate transaction, for example by using `REQUIRES_NEW` for the risky operation. This is a common issue in batch processing where some records fail but others should still be processed.

S21. A distributed system uses `@CacheEvict(allEntries = true)` after every small update, causing cache thrashing. How do you improve it?

Answer: Evicting the entire cache for every small update defeats the purpose of caching because it constantly invalidates useful entries and forces the database to handle all reads. I switch to targeted eviction using the specific key that changed, for example `@CacheEvict(value = "products", key = "#productId")`. If the update affects a computed list, I maintain a separate cache key for that list and evict only that key. For complex relationships, I use multiple cache names and evict only the relevant ones. In some cases, I use `@CachePut` to refresh a single entry rather than evicting it. The goal is to keep as much valid data in the cache as possible while removing only what is stale.

S22. A `@Transactional` method on a controller does not roll back. Why?

Answer: `@Transactional` is typically meant for service-layer methods. Controllers are proxied by Spring MVC, but the transaction annotation may not take effect if the controller is not a Spring bean, if the method is not public, or if the controller is called outside the Spring-managed request flow. More importantly, controller methods often do database operations through a service that is already transactional, and adding `@Transactional` on the controller is not the right place. The best practice is to keep transaction boundaries in the service layer. I move the annotation to the service method, ensure the method is public, and verify that the service bean is obtained from the Spring context rather than instantiated directly.

S23. A cache key built from a user object does not work because the key changes after serialization. How do you fix it?

Answer: If the cache key is derived from an object's `toString()` or default hash code, and the object contains fields that change after serialization, the key becomes inconsistent. I build the key only from stable identifiers, such as the user ID, and avoid using mutable objects or the entire object as the key. For example, `@Cacheable(value = "preferences", key = "#user.id")` is correct, while `key = "#user"` is risky. If multiple arguments are needed, I concatenate their IDs with a separator. I also ensure that the same key builder is used for both `@Cacheable` and `@CacheEvict` so reads and writes refer to the same cache entry.

S24. A database write under `@Transactional` appears successful, but the data is not persisted. Why?

Answer: The most common cause is that the transaction manager used by `@Transactional` does not match the one used by the repository. For example, if there are multiple datasources and the annotation is using the default transaction manager while the repository writes to a secondary datasource, the transaction may not cover the write. Another cause is that the entity is detached and `merge` or `save` is not called. I verify the transaction manager configuration, check that the repository and service use the same `EntityManager` or `DataSource`, and ensure the method is called through the proxy. I also check whether the method is marked `readOnly = true`, which can prevent flush and commit in some configurations.

S25. A team wants to use Redis for caching but also for session storage and rate limiting. How do you manage the cache namespace?

Answer: Using Redis for multiple concerns requires clear namespacing to prevent key collisions and to allow different TTL and eviction policies per concern. I use separate Redis databases or, better, key prefixes such as `cache:`, `session:`, and `ratelimit:` so each concern has its own namespace. Spring Cache uses cache names as part of the key, so I give each cache a descriptive name. I also configure separate serializers and TTLs per cache using `RedisCacheConfiguration` and `RedisCacheManagerBuilderCustomizer`. For sessions, I use Spring Session's `RedisIndexedSessionRepository` with its own prefix. Monitoring tools like Redis MONITOR or keyspace notifications help verify that keys are separated correctly.